CS611 MACHINE LEARNING ENGINEERING · ASSIGNMENT 1

Credit Scoring ML Pipeline

Data pipeline design and implementation (based on the Medallion Architecture)

Call for feedback/ comments on bronze, silver and gold tables on the credit scoring model

For management, data engineers and business users · September 2026

EXECUTIVE SUMMARY

Call for feedback/ comments on bronze/ silver/ gold tables (label/ feature) before feeding in the model

The feature table joins to the label store at the gold tables.

12,500
loan applications, one row each

57
model features from 3 raw sources

8,974
labelled loans, 28.9% defaulted 30DPD6M
(30+ days past due at month 6)

100%
labelled loans match a feature row on loan_id

DESIGN PRINCIPLES

BRONZE

Keep it raw

Every column stored as values for replicability and audit.

SILVER

Clean, don’t guess

Types enforced.
Impossible values become null.

GOLD

One row per application

Three sources combined into one table that joins to the label store.

LEAKAGE ADDRESSED

Safe by design

Only data known at the application date. No imputing or scaling before training.

EXPLORATORY DATA ANALYSIS

Four raw files feed the store: two at application grain, two monthly

01

feature_attributes

One row per customer, at loan application

12,500

rows · 12,500 customers
Jan 2023 to Jan 2025

- Age and Occupation.
- Age has typos and impossible values.
- PII are Name and SSN.

02

feature_financials

One row per customer, at loan application

12,500

rows · 12,500 customers
Jan 2023 to Jan 2025

- 21 credit and income columns.
- Text fields polluted with placeholders, extreme outliers.

03

feature_clickstream

One row per customer per month

215,376

rows · 8,974 customers
Jan 2023 to Dec 2024

- 20 anonymous integer signals, no nulls.
- 24 monthly snapshots each, most after the application date.

04

lms_loan_daily

One row per loan per month

137,500

rows · 12,500 customers
Jan 2023 to Nov 2025

- Loan repayments.
- Source of the default label.
- One 10-month loan per customer.

For attributes and financials, snapshot_date is the loan start date, so each row describes the customer on the day they applied (100% match to the loan system).

Five kinds of bad data were found, and treatment fixes on silver

Issue	Example	Where (rows affected)	Treatment
Stray underscores on numbers	"52312.68_", "40_"	Annual_Income (859), Num_of_Loan (623), Num_of_Delayed_Payment (374), Outstanding_Debt (139), Age	Strip the underscore, then cast to the declared type
Placeholder text	"_", "!@9#%8", "_______"	Credit_Mix (2,611), Payment_Behaviour (998), Occupation (880), Changed_Credit_Limit (254)	Keep only known categories, everything else becomes null
Sentinel and impossible numbers	Age -500 or 8,678; 10000; −3.3e26	Age, income, accounts, cards, rates, loans, delays, inquiries, EMI, invested, balance	Valid-range check, out-of-range becomes null (next slide)
Personal data	SSN "#F%$D@*&8", customer names	Name and SSN in attributes	Dropped in silver: no predictive value, should not travel downstream
Values packed into text	"10 Years and 9 Months"; "Credit-Builder Loan, and Home Equity Loan"	Credit_History_Age, Type_of_Loan	Months as an integer in silver; one flag per loan type in gold

Rule of thumb: a bad value becomes null. The model pipeline decides how to fill gaps, using training data only.

Range limits null out under 5% of any column, and no loan rows are deleted

Column	Valid range	Raw extremes	Set to null
Age	14 to 100	-500 to 8,678	319
Annual_Income	0 to 200,000	up to 23.8 million	114
Num_Bank_Accounts	0 to 10	-1 to 1,756	174
Num_Credit_Card	0 to 11	up to 1,499	296
Interest_Rate	0 to 34	up to 5,789	270
Num_of_Loan	0 to 9	-100 to 1,495	**567**
Num_of_Delayed_Payment	0 to 30	-3 to 4,293	189
Num_Credit_Inquiries	0 to 17	up to 2,554	195
Total_EMI_per_month	0 to 2,000	up to 81,971	408
Amount_invested_monthly	0 to 5,000	placeholder 10,000	**558**
Monthly_Balance	0 to 100,000	-3.3e26 (one row)	1

How the limits were set

Each limit sits at a natural gap: values run smoothly up to the limit, then jump to hundreds or thousands.

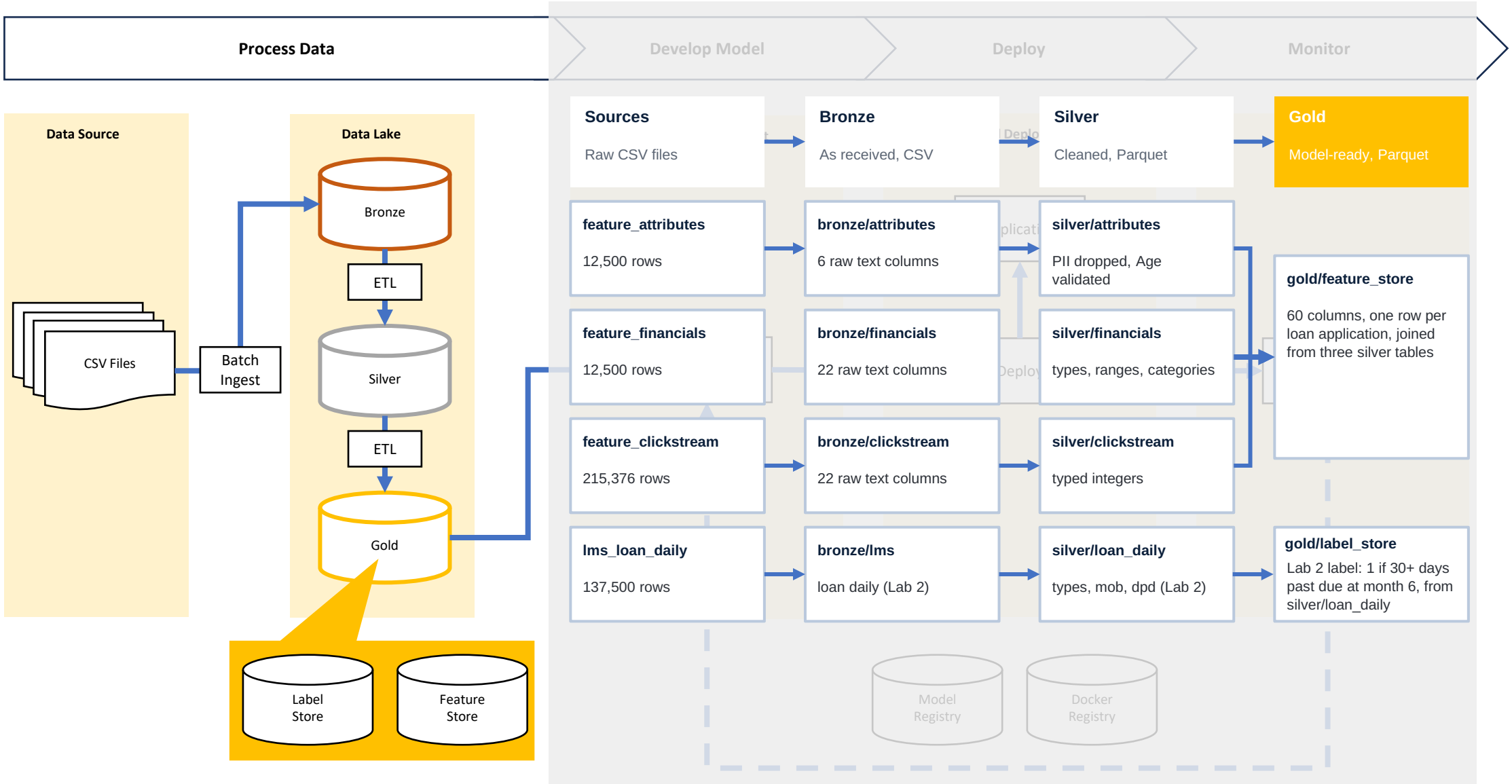
Why null, not cap or delete

Capping invents data and deleting rows loses loans. No column loses more than 4.5% of its values.

TO CONFIRM WITH BUSINESS TEAM

Limits are data-driven. Product owners should confirm them, for example the minimum applicant age.

Proposed High-Level ML Pipeline (Working)



GOLD FEATURE STORE

The gold table has 60 columns: 3 keys and 57 features, 15 of them engineered

Feature group	Columns	Count	Source
Keys	loan_id, Customer_ID, snapshot_date	3	Application
Applicant profile	Age, Occupation	2	Attributes
Credit and income	Annual_Income, Monthly_Inhand_Salary, Outstanding_Debt, Credit_Mix, Credit_History_Months and 14 more	19	Financials
Loan mix flags	One 0/1 flag for each of 9 loan types, from the Type_of_Loan text	9	Engineered
Payment behaviour	spend_level (Low/High), payment_size (Small/Medium/Large)	2	Engineered
Affordability ratios	debt_to_income, emi_to_salary, invested_to_salary, balance_to_salary	4	Engineered
Clickstream	fe_1 to fe_20, has_clickstream	21	Clickstream

GRAIN

One row per loan application, 12,500 rows. loan_id looks like CUS_0x1000_2023_05_01 and matches the label store.

LEFT OUT ON PURPOSE

Scaling, imputing and encoding categories. They learn from data, so they belong in the training pipeline.

DATA LEAKAGE

Four leakage risks are each closed by a control inside the pipeline

	HOW IT COULD HAPPEN HERE	CONTROL IN THE PIPELINE
Target leakage	Loan system columns (paid amount, overdue amount, days past due) contain the answer	No loan system column is in the feature store. The label sits in its own table and joins only by loan_id.
Temporal leakage	Clickstream has 24 monthly snapshots per customer, most dated after the application	Gold reads only the partition of the application date. Later months never enter the row.
Train-test contamination	Scaling or imputing on all rows lets test data shape the training data	Nothing is fitted in the pipeline. Nulls stay null and the model pipeline fits on training data only.
Label timing	The default label is observed 6 months after application	Checked: label date minus 6 months equals the application date for all 8,974 labelled loans.

For the modelling step: split train and test by application date (older loans train, newer loans test), not at random.

Two gaps remain: 28% of applicants lack clickstream and 21% lack Credit_Mix

MISSING CLICKSTREAM

3,526 applicants (28%) have no clickstream

Finding: They are from Jul 2024 to Jan 2025, which have no label yet. All 8,974 labelled loans have clickstream.

Concern? Not for training. Yes for scoring: new applicants would arrive without fe_1 to fe_20.

Action. Ask data engineering if clickstream is delayed or dropped. Compare a model without it. Decision not to use has_clickstream as a feature.

MISSING CREDIT_MIX

2,611 applicants (21%) have no Credit_Mix

Finding. Nulls default at 28.0% against 28.9% overall, so missing looks random. Known values are strong: Good 16%, Standard 24%, Bad 56%.

Concern? Low. The column is valuable and the gap is not hiding a pattern.

Action. Keep the column. Treat null as its own Unknown group instead of dropping rows.

NEXT STEPS

1. Confirm range limits with business owners.
2. Settle the clickstream question.
3. Split by application date.
4. Fit imputers and encoders on training data only.

TARGET ML PIPELINE

Phase 1 is built; phases 2 to 4 are the proposed path to a monitored model

PHASE 1

Process data

Current build

- Bronze, silver and gold tables in PySpark and Docker
- Gold feature store with 57 features
- Label store: 30+ days past due at month 6
- Leakage and range checks built in

→

PHASE 2

Develop model

Proposed

- Split train and test by application date
- Impute and encode on training data only
- Logistic regression baseline, then gradient boosting
- Track every run, register the best model

→

PHASE 3

Deploy model

Proposed

- Package model and preprocessing as one versioned unit
- Score new applications in a monthly batch job
- Save predictions to a new gold table
- Run champion and challenger before switching

→

PHASE 4

Monitor model

Proposed

- Data drift on features and null rates
- Prediction drift every month
- Real performance once labels mature, 6 months later
- Alerts and retraining triggers

FEEDBACK LOOP

Drift or weaker performance sends phase 4 back to phase 2, retraining on the newest gold tables.

ACROSS ALL PHASES

Monthly scheduled runs, code and versions in GitHub, Docker for identical environments.